Flow Matching: Complete Reading Guide

For Upgrading ToyDiffusion from DDPM to Flow Matching

Compiled for offline study

February 2026

Abstract

This document is a self-contained reference for understanding how to upgrade ToyDiffusion—a toy DDPM implementation that generates 2D Homer Simpson scatter plots—from the classic **DDPM** framework (Ho et al., 2020) to the modern **Flow Matching** paradigm (Lipman et al., 2022; Liu et al., 2022). All key equations are typeset in full, derivations are expanded, and a side-by-side comparison makes the correspondence explicit. Stable Diffusion 3, Flux, and most modern production generative models have switched from DDPM to flow matching.

Contents

1 The Big Picture **3**
1.1 What DDPM Does (Your Original Code) . . . 3
1.2 What Flow Matching Does . . . 3

2 Paper 1: Flow Matching for Generative Modeling **3**
2.1 Density Functions vs. Samples: A Key Distinction . . . 3
2.2 Continuous Normalizing Flows . . . 4
2.3 The Continuity Equation . . . 5
2.4 The Flow Matching Loss (Intractable) . . . 5
2.5 Conditional Flow Matching (The Key Contribution) . . . 6
2.6 Why Optimal Transport Paths? . . . 7
2.7 Connection to Diffusion Models . . . 7

3 Paper 2: Rectified Flow **7**
3.1 Core Formulation . . . 7
3.2 The Reflow Procedure (Unique Contribution) . . . 8
3.3 Theoretical Guarantees . . . 8
3.4 Few-Step and One-Step Generation . . . 8

4 DDPM vs. Flow Matching: Side-by-Side **8**
4.1 Notation Comparison . . . 8
4.2 Process Comparison . . . 9
4.3 Equation Comparison . . . 9

5 The Math You Need **10**
5.1 Continuous Normalizing Flows and the Continuity Equation . . . 10
5.2 Gaussian Conditional Probability Paths . . . 10
5.3 Deriving the Conditional Velocity Field . . . 10
5.4 Score–Velocity–Epsilon Conversion . . . 11

6 The 3 Changes to Your Code **11**
6.1 Change 1: Replace `q_sample` with Linear Interpolation . . . 11
6.2 Change 2: Training Target → Velocity . . . 12
6.3 Change 3: Sampling → Euler ODE Integration . . . 12
6.4 Complete Training Algorithm . . . 13
6.5 Complete Sampling Algorithm . . . 13

7 Advanced: Reflow for One-Step Generation **13**
7.1 Generating Coupled Pairs . . . 13
7.2 Retraining on Coupled Pairs . . . 14
7.3 One-Step Generation After Reflow . . . 14

8 Reading Order and Resources **14**
8.1 Recommended Reading Order . . . 14
8.2 Code Repositories . . . 15

Flow Matching Cheat Sheet **16**

References **16**

1 The Big Picture

1.1 What DDPM Does (Your Original Code)

Your ToyDiffusion project implements **DDPM** (Ho et al., 2020):

1. Define a **Markov chain** that adds Gaussian noise over T discrete steps.
2. Train a neural network to **reverse** the process step by step.
3. The model predicts either the noise ϵ or the posterior mean $\tilde{\mu}_t$.

The forward process gradually destroys the data signal:

$$q(x_t \mid x_0) = \mathcal{N}(x_t;\ \sqrt{\bar{\alpha}_t}\, x_0,\ (1 - \bar{\alpha}_t)\, \mathbf{I})$$

and the reverse process learns to undo it, one step at a time.

1.2 What Flow Matching Does

Flow Matching (2022) provides a simpler alternative:

1. Define a **continuous path** from noise to data via linear interpolation.
2. Train a neural network to predict the **velocity field** $v_\theta(x, t)$ along that path.
3. Generate samples by integrating a simple ODE.

Key Insight

Diffusion models are a *special case* of flow matching with a particular (suboptimal) choice of interpolation path. Using straight-line (Optimal Transport) paths instead gives straighter trajectories, which means fewer sampling steps and simpler math.

2 Paper 1: Flow Matching for Generative Modeling

Authors: Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, Matt Le (Meta FAIR)
Published: October 2022 (ICLR 2023)
arXiv: https://arxiv.org/abs/2210.02747

2.1 Density Functions vs. Samples: A Key Distinction

Before diving into the math, we need to be precise about one thing. A **probability density function** $p(x)$ is a mathematical function $p : \mathbb{R}^d \to \mathbb{R}$ satisfying $\int p(x)\, dx = 1$. It tells you "how likely is x?" A **sample** (or realization) is an actual point $x \in \mathbb{R}^d$ drawn from that distribution.

In your ToyDiffusion project:

- **The Gaussian prior** $p_0 = \mathcal{N}(0, \mathbf{I})$: we have the exact formula $p_0(x) = \frac{1}{(2\pi)^{d/2}} \exp\!\left(-\frac{1}{2}\|x\|^2\right)$, *and* we can generate samples via `torch.randn(N, 2)`.
- **Homer Simpson** p_{data}: we do **not** have a formula. We only have samples—the (x, y) scatter plot points from your dataset. But mathematically, some unknown density function $p_{\text{data}}(x)$ exists that is high where Homer's features are and low everywhere else.

- **Intermediate distributions** p_t (for $0 < t < 1$): we have **neither** the formula **nor** pre-computed samples. We *could* generate samples at time t by flowing p_0 samples forward, but we never write down $p_t(x)$ as a closed-form expression.

Watch Out

The math in this document talks about density functions $p_t(x)$, but your code only ever works with **samples** (actual tensors of points). The entire point of Conditional Flow Matching (Section 2.5) is to set up a training loss that only requires samples from p_0 and p_{data}—never p_t itself.

2.2 Continuous Normalizing Flows

A **Continuous Normalizing Flow** (CNF) defines a time-dependent ODE whose solution transforms a simple prior $p_0 = \mathcal{N}(0, \mathbf{I})$ into the data distribution $p_1 \approx p_{\text{data}}$:

$$\frac{dx}{dt} = v_\theta(x, t), \qquad t \in [0, 1]. \tag{1}$$

What "integrating this ODE" means. Recall that solving any ODE requires two things:

1. The differential equation itself: $\frac{dx}{dt} = v_\theta(x, t)$
2. An **initial condition**: $x(0) = x_0$

Given an initial sample $x_0 \sim \mathcal{N}(0, \mathbf{I})$, the solution at time t=1 is:

$$x_1 = x_0 + \int_0^1 v_\theta(x(t),\, t)\, dt \tag{2}$$

Note: $\int_0^1 \frac{dx}{dt}\, dt = x(1) - x(0) = x_1 - x_0$ gives you the *change* in x, not x_1 itself. You always need the initial condition x_0 to recover x_1. In practice this integral is approximated by Euler steps (see Section 6):

$$x_{t+\Delta t} \approx x_t + v_\theta(x_t, t) \cdot \Delta t \tag{3}$$

The flow map. The ODE defines a **flow map** $\phi_t : \mathbb{R}^d \to \mathbb{R}^d$—a function that tells you where each point ends up at time t:

$$\phi_0(x) = x \quad \text{(identity)}, \qquad \frac{d}{dt}\phi_t(x) = v_\theta(\phi_t(x),\, t).$$

If you start at x_0, then $\phi_t(x_0)$ is your position at time t.

The pushforward (what $[\phi_t]_\# p_0$ means). This notation looks scary but the idea is simple: take every point sampled from p_0, apply the map ϕ_t to each point, and look at the distribution of the results. That resulting distribution is the **pushforward**:

$$p_t = [\phi_t]_\# p_0. \tag{4}$$

In code, this is exactly what your ToyDiffusion visualization already does:

```
# Sample 1000 points from p_0
x = torch.randn(1000, 2) # these are distributed as p_0

# Flow them forward to time t using the ODE
for step in range(n_steps_to_t):
```

```
    x = x + v_theta(x, t) * dt # Euler integration

# Now x is distributed as p_t = [phi_t]_# p_0
plot(x) # visualize the pushforward!
```

When you animated particles flowing from noise to Homer Simpson, you were computing the pushforward at each frame.

2.3 The Continuity Equation

The probability path p_t and velocity field u_t must satisfy the **continuity equation**. Before stating it, let's unpack the notation.

Notation: u_t is the velocity field. The papers use $u_t(x)$ as their notation for the velocity field. This is the same object as $v_\theta(x,t)$ (what your code will call it)—just different notation. Don't let the symbol change confuse you: $u_t(x) = v_\theta(x,t) = v(x,t)$ all refer to the same thing.

What is divergence? The divergence $\mathrm{div}(\cdot)$ measures *how much a vector field is spreading out or converging* at a point. For a 2D velocity field $v = (v_x, v_y)$:

$$\mathrm{div}(v) = \frac{\partial v_x}{\partial x} + \frac{\partial v_y}{\partial y}$$

- $\mathrm{div} > 0$: particles are spreading apart (like a source / explosion)
- $\mathrm{div} < 0$: particles are converging (like a sink / drain)
- $\mathrm{div} = 0$: the flow preserves volume (incompressible)

The equation. Now we can state it:

$$\underbrace{\frac{\partial p_t}{\partial t}}_{\substack{\text{how fast density}\\ \text{changes at } x}} + \underbrace{\mathrm{div}(p_t \cdot u_t)}_{\substack{\text{how much probability}\\ \text{mass flows away from } x}} = 0. \tag{5}$$

The product $p_t \cdot u_t$ is the **probability flux**: density times velocity = "flow of probability mass." The equation says these two terms must balance: if probability mass is flowing away from a region ($\mathrm{div}(p_t \cdot u_t) > 0$), the density there must decrease ($\frac{\partial p_t}{\partial t} < 0$), and vice versa.

Key Insight

This is just the law of conservation of probability mass. Probability can't be created or destroyed, only moved around. If particles leave a region, density decreases there. If particles enter, density increases. The continuity equation is the mathematical statement of this obvious physical fact.

2.4 The Flow Matching Loss (Intractable)

The ideal objective would be to match the velocity field directly:

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}_{t\sim U(0,1),\, x\sim p_t} \left\| v_\theta(x,t) - u_t(x) \right\|^2 \tag{6}$$

where $u_t(x)$ is the true velocity field that generates the probability path p_t.

Watch Out

This loss is **intractable**: we don't have access to $p_t(x)$ or $u_t(x)$ in closed form, because they involve integrals over all data points. Remember from Section 2.1: p_t is a theoretical density we never compute! We can't sample from it, and we don't know u_t. This is why we need the conditional trick below.

2.5 Conditional Flow Matching (The Key Contribution)

Instead of working with intractable marginals, we define **conditional** probability paths. For each data point $x_1 \sim p_{\text{data}}$, the conditional probability path is:

$$p_t(x \mid x_1) = \mathcal{N}(x;\ \mu_t(x_1),\ \sigma_t(x_1)^2\,\mathbf{I}) \tag{7}$$

with boundary conditions:

$$t = 0: \quad \mu_0 = 0, \quad \sigma_0 = 1 \quad \text{(standard Gaussian prior)} \tag{8}$$

$$t = 1: \quad \mu_1 = x_1, \quad \sigma_1 \approx 0 \quad \text{(concentrated at data point)} \tag{9}$$

For the **Optimal Transport (OT) linear path**:

$$\mu_t(x_1) = t \cdot x_1, \qquad \sigma_t = 1 - t \tag{10}$$

Sampling from this conditional path is equivalent to:

$$\boxed{x_t = (1-t)\,x_0 + t\,x_1} \qquad \text{where } x_0 \sim \mathcal{N}(0, \mathbf{I}) \tag{11}$$

The conditional velocity field for this path is:

$$u_t(x \mid x_1) = \frac{\sigma_t'(x_1)}{\sigma_t(x_1)}(x - \mu_t(x_1)) + \mu_t'(x_1) \tag{12}$$

Derivation. Taking the time derivative of $x_t = (1-t)\,x_0 + t\,x_1$:

$$\frac{dx_t}{dt} = -x_0 + x_1 = x_1 - x_0 \tag{13}$$

So the conditional velocity for the OT path simplifies to the remarkably simple expression:

$$\boxed{u_t(x \mid x_1) = x_1 - x_0} \tag{14}$$

A constant! The velocity doesn't depend on x or t—it's simply the straight-line direction from noise to data.

The **Conditional Flow Matching (CFM)** loss is then:

$$\boxed{\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t\sim U(0,1),\ x_0\sim\mathcal{N}(0,\mathbf{I}),\ x_1\sim p_{\text{data}}} \left\| v_\theta(x_t, t) - (x_1 - x_0) \right\|^2} \tag{15}$$

Key Insight

Key Theorem (Lipman et al., 2022): The gradients of $\mathcal{L}_{\text{CFM}}$ and $\mathcal{L}_{\text{FM}}$ are identical:

$$\nabla_\theta\, \mathcal{L}_{\text{CFM}}(\theta) = \nabla_\theta\, \mathcal{L}_{\text{FM}}(\theta).$$

Training with the tractable conditional loss is *equivalent* to training with the intractable marginal loss. This is the central result that makes flow matching practical.

Proof sketch. The marginal velocity field decomposes as:

$$u_t(x) = \frac{\int u_t(x \mid x_1)\, p_t(x \mid x_1)\, q(x_1)\, dx_1}{\int p_t(x \mid x_1)\, q(x_1)\, dx_1} = \mathbb{E}_{q(x_1|x)}\big[u_t(x \mid x_1)\big]$$

Expanding the FM loss and applying the tower property of conditional expectations shows the gradient equivalence. The full proof relies on exchanging the order of integration over x and x_1.

2.6 Why Optimal Transport Paths?

The linear interpolation $x_t = (1-t)\,x_0 + t\,x_1$ is the **Wasserstein-2 optimal transport geodesic** between the conditional distributions $\mathcal{N}(0, \mathbf{I})$ and δ_{x_1}.

This has two important consequences:

1. The marginal velocity field is smoother and closer to straight.
2. Fewer ODE solver steps are needed at inference (even 10–20 Euler steps work well).

Compare: DDPM-style paths are curved (they first destroy the signal heavily, then denoise), requiring ∼1000 steps or specialized samplers like DDIM.

2.7 Connection to Diffusion Models

Diffusion models correspond to flow matching with a **specific non-linear path**:

$$x_t = \sqrt{\bar{\alpha}_t}\, x_1 + \sqrt{1-\bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, \mathbf{I}) \tag{16}$$

This is the Variance Preserving SDE (VP-SDE) schedule. In the flow matching framework, this corresponds to:

$$\mu_t(x_1) = \sqrt{\bar{\alpha}_t}\, x_1, \qquad \sigma_t = \sqrt{1-\bar{\alpha}_t}$$

The score function and velocity field are related by:

$$v(x,t) = f(x,t) - \tfrac{1}{2}\, g(t)^2\, \nabla_x \log p_t(x) \tag{17}$$

where f and g are the drift and diffusion coefficients of the corresponding SDE formulation $dx = f(x,t)\,dt + g(t)\,dW_t$.

3 Paper 2: Rectified Flow

Authors: Xingchao Liu, Chengyue Gong, Qiang Liu (UT Austin)
Published: September 2022 (ICLR 2023 Spotlight)
arXiv: https://arxiv.org/abs/2209.03003

3.1 Core Formulation

Same ODE setup: learn v_θ such that

$$\frac{dZ_t}{dt} = v_\theta(Z_t, t), \qquad t \in [0,1]. \tag{18}$$

The training objective is identical to Conditional Flow Matching:

$$\mathcal{L} = \mathbb{E}_{t\sim U(0,1)}\, \mathbb{E}_{X_0\sim\mathcal{N}(0,\mathbf{I}),\, X_1\sim p_{\text{data}}}\, \|v_\theta(X_t,t) - (X_1 - X_0)\|^2 \tag{19}$$

where $X_t = (1-t)\,X_0 + t\,X_1$.

3.2 The Reflow Procedure (Unique Contribution)

The key idea: **iteratively straighten the flow trajectories**.

Round 1 (Initial Training). Sample independent pairs (X_0, X_1) where $X_0 \sim \mathcal{N}(0, \mathbf{I})$ and $X_1 \sim p_{\text{data}}$. Train v_1 with the standard loss (19).

Round 2 (Reflow). For many samples $X_0 \sim \mathcal{N}(0, \mathbf{I})$, solve the ODE of v_1 forward to obtain:

$$Z_1 = X_0 + \int_0^1 v_1(Z_t, t)\, dt \tag{20}$$

Now (X_0, Z_1) are **coupled** pairs—they lie on the same learned trajectory. Retrain a new model v_2 on these coupled pairs:

$$\mathcal{L}_{\text{reflow}} = \mathbb{E}_t \left\| v_2\big((1-t)\, X_0 + t\, Z_1,\ t\big) - (Z_1 - X_0) \right\|^2 \tag{21}$$

Why it works. In round 1, independent pairs produce crossing linear interpolation paths, forcing the velocity field to average at intersections (creating curvature). After reflow, the pairs are correlated—their interpolations cross much less, so the retrained flow is straighter.

3.3 Theoretical Guarantees

The straightness of the flow is measured by:

$$S(v) = \int_0^1 \mathbb{E}\Big[\|v(Z_t, t) - (Z_1 - Z_0)\|^2 \Big]\, dt \tag{22}$$

- $S(v) = 0$ if and only if all trajectories are perfectly straight lines.
- Each round of reflow **monotonically decreases** $S(v)$, i.e. $S(v_{k+1}) \leq S(v_k)$.
- In the limit, $S(v) \to 0$ and all trajectories become straight.

3.4 Few-Step and One-Step Generation

If all trajectories are perfectly straight, a single Euler step suffices:

$$Z_1 = Z_0 + v(Z_0, 0) \qquad \text{(one-step generation!)} \tag{23}$$

In practice:

- After 1 reflow: good results with 2–4 Euler steps.
- After 2 reflows: competitive 1-step generation.

4 DDPM vs. Flow Matching: Side-by-Side

4.1 Notation Comparison

Watch Out

The time direction is **flipped**! In DDPM, t=0 is clean data and t=T is noise. In flow matching, t=0 is noise and t=1 is data.

Concept	DDPM (Your Code)	Flow Matching
Time domain	$t \in \{0, 1, \ldots, T\}$ discrete	$t \in [0, 1]$ continuous
Data	x_0 (original, clean)	x_1 (target at t=1)
Noise	x_T (fully noised)	x_0 (noise at t=0)
Direction	noise $\to$ data: t: $T \to 0$	noise $\to$ data: t: $0 \to 1$

4.2 Process Comparison

Aspect	DDPM (Your Original)	Flow Matching
Forward process	Markov chain, T discrete steps	Linear interpolation, continuous
Noise schedule	Required (β_t, α_t, $\bar{\alpha}_t$)	**Not needed**
Training target	Predict ϵ or $\tilde{\mu}_t$	Predict velocity $v = x_1 - x_0$
Loss function	$\|\tilde{\mu}_t - \mu_\theta\|^2$	$\|v_{\text{target}} - v_\theta\|^2$
Sampling	~1000 steps, complex posterior	~10–100 Euler steps: $x \mathrel{+}= v \cdot dt$
Math complexity	KL divergence, posterior distributions	Simple ODE integration

4.3 Equation Comparison

Interpolation (creating a noisy sample at time t)

DDPM

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

Flow Matching

$$x_t = (1 - t)\, x_0 + t\, x_1, \qquad x_0 \sim \mathcal{N}(0, \mathbf{I})$$

Training target (what the network predicts)

DDPM

μ-prediction: a complex function of x_0, x_t, α_t, β_t:

$$\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\, \beta_t}{1 - \bar{\alpha}_t}\, x_0 + \frac{\sqrt{\alpha_t}\, (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}\, x_t$$

ϵ-prediction: the noise that was added: target $= \epsilon$.

Flow Matching

Velocity prediction: the direction from noise to data:

$$v_{\text{target}} = x_1 - x_0$$

Sampling (generating new data)

DDPM

Reverse Markov chain:

$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t\, z, \qquad z \sim \mathcal{N}(0, \mathbf{I})$$

Flow Matching

Euler ODE step:

$$x_{t+\Delta t} = x_t + v_\theta(x_t, t) \cdot \Delta t$$

5 The Math You Need

5.1 Continuous Normalizing Flows and the Continuity Equation

A CNF defines a velocity field $v : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d$ whose ODE (1) has the unique solution flow ϕ_t. The induced probability path p_t satisfies the continuity equation (5).

The **change of variables formula** for CNFs gives the exact log-likelihood:

$$\log p_1(x_1) = \log p_0(\phi_1^{-1}(x_1)) - \int_0^1 \operatorname{div}(v(\phi_t(x), t))\, dt \tag{24}$$

This is important theoretically, though in practice flow matching sidesteps likelihood computation by using the regression loss (15) directly.

5.2 Gaussian Conditional Probability Paths

The general Gaussian conditional path is:

$$p_t(x \mid x_1) = \mathcal{N}(x;\ \mu_t(x_1),\ \sigma_t(x_1)^2\, \mathbf{I}) \tag{25}$$

with boundary conditions:

$$t = 0: \qquad \mu_0 = 0, \quad \sigma_0 = 1 \quad \text{(standard Gaussian prior)} \tag{26}$$

$$t = 1: \qquad \mu_1 = x_1, \quad \sigma_1 \approx 0 \quad \text{(concentrated at data)} \tag{27}$$

Two important choices:

Path	$\mu_t(x_1)$	σ_t
OT / Linear (recommended)	$t \cdot x_1$	$1 - t$
VP / Diffusion (DDPM-like)	$\sqrt{\bar{\alpha}_t}\, x_1$	$\sqrt{1 - \bar{\alpha}_t}$

Sampling from either path follows:

$$x_t = \mu_t(x_1) + \sigma_t \cdot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, \mathbf{I}) \tag{28}$$

For the OT path: $x_t = t\, x_1 + (1-t)\, \epsilon = (1-t)\, x_0 + t\, x_1$ (with $x_0 = \epsilon$).

5.3 Deriving the Conditional Velocity Field

General case. Given the Gaussian conditional path $p_t(x \mid x_1) = \mathcal{N}(x;\ \mu_t, \sigma_t^2\, \mathbf{I})$, we can write $x_t = \mu_t + \sigma_t\, \epsilon$ where $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ is fixed. Taking the time derivative:

$$u_t(x \mid x_1) = \frac{dx_t}{dt} = \mu_t' + \sigma_t'\, \epsilon = \mu_t' + \frac{\sigma_t'}{\sigma_t}\,(x_t - \mu_t) \tag{29}$$

where in the last step we used $\epsilon = (x_t - \mu_t)/\sigma_t$.

OT/Linear path. With $\mu_t = t\,x_1$ and $\sigma_t = 1 - t$, we get $\mu_t' = x_1$ and $\sigma_t' = -1$, so:

$$\begin{aligned} u_t(x \mid x_1) &= x_1 + \frac{-1}{1-t}\,(x_t - t\,x_1) \\ &= x_1 - \frac{x_t - t\,x_1}{1-t} \\ &= x_1 - \frac{(1-t)\,x_0 + t\,x_1 - t\,x_1}{1-t} \quad \text{(substituting } x_t = (1-t)x_0 + t\,x_1\text{)} \\ &= x_1 - x_0 \end{aligned} \tag{30}$$

VP/Diffusion path. With $\mu_t = \sqrt{\bar{\alpha}_t}\,x_1$ and $\sigma_t = \sqrt{1-\bar{\alpha}_t}$:

$$u_t^{\mathrm{VP}}(x \mid x_1) = \frac{d\sqrt{\bar{\alpha}_t}}{dt}\,x_1 + \frac{d\sqrt{1-\bar{\alpha}_t}/dt}{\sqrt{1-\bar{\alpha}_t}}\,(x - \sqrt{\bar{\alpha}_t}\,x_1) \tag{31}$$

This is significantly more complex, depending on the specific $\bar{\alpha}_t$ schedule.

5.4 Score–Velocity–Epsilon Conversion

Given a general schedule α_t (not to be confused with $\bar{\alpha}_t$ in DDPM notation), where $x_t = \alpha_t\,x_1 + \sigma_t\,\epsilon$, the three prediction targets are related:

$$\text{Velocity:} \quad v = \alpha_t'\,x_1 + \sigma_t'\,\epsilon \tag{32}$$

$$\text{Score:} \quad \nabla_x \log p_t(x) = -\frac{x - \alpha_t\,x_1}{\sigma_t^2} = -\frac{\epsilon}{\sigma_t} \tag{33}$$

$$\text{Epsilon:} \quad \epsilon = \frac{x_t - \alpha_t\,x_1}{\sigma_t} \tag{34}$$

$$\text{Data:} \quad x_1 = \frac{x_t - \sigma_t\,\epsilon}{\alpha_t} \tag{35}$$

You can convert between any two given the schedule (α_t, σ_t).

Velocity from epsilon (for OT path, $\alpha_t = t$, $\sigma_t = 1 - t$):

$$v = \alpha_t'\,x_1 + \sigma_t'\,\epsilon = 1 \cdot x_1 + (-1) \cdot \epsilon = x_1 - \epsilon = x_1 - x_0 \tag{36}$$

Velocity from epsilon (for VP path):

$$v = \frac{d\sqrt{\bar{\alpha}_t}}{dt}\,x_1 - \frac{d\sqrt{1-\bar{\alpha}_t}}{dt}\,\epsilon \tag{37}$$

6 The 3 Changes to Your Code

6.1 Change 1: Replace `q_sample` with Linear Interpolation

DDPM

Old (`diffusion.py:q_sample`):

```
# x_t = sqrt(alpha_bar_t) * x_0 + sqrt(1 - alpha_bar_t) * eps
alpha_bar_t = list_bar_alphas[t]
mean = alpha_bar_t * x_start
cov = torch.eye(x_start.shape[0]) * (1 - alpha_bar_t)
x_t = MultivariateNormal(mean, cov).sample()
```

Flow Matching

New (flow matching interpolation):

```
def interpolate(x_data, t):
    x_noise = torch.randn_like(x_data) # x_0 ~ N(0, I)
    x_t = (1 - t) * x_noise + t * x_data # Eq. (6)
    v_target = x_data - x_noise # Eq. (9)
    return x_t, v_target
```

No more $\bar{\alpha}_t$, α_t, or β_t schedules!

6.2 Change 2: Training Target → Velocity

DDPM

Old (trains μ_θ against $\tilde{\mu}_t$):

```
mu_theta = model(x_t, t)
mu_tilde, _ = posterior_q(x_start, x_t, t, ...)
loss = MSE(mu_tilde, mu_theta)
```

Flow Matching

New (trains v_θ against $v_{\text{target}} = x_1 - x_0$):

```
v_pred = model(x_t, t)
loss = MSE(v_target, v_pred)
```

6.3 Change 3: Sampling → Euler ODE Integration

DDPM

Old (complex reverse Markov chain):

```
# For t = T, T-1, ..., 1:
mu_theta = model(x_t, t)
x_{t-1} ~ N(mu_theta, beta_t * I)
```

Flow Matching

New (simple Euler integration):

```
x = torch.randn(n_samples, dim) # start from noise
dt = 1.0 / n_steps
for i in range(n_steps):
    t = i * dt
    v = model(x, t)
    x = x + v * dt # Euler step
# x is now a generated sample
```

6.4 Complete Training Algorithm

Algorithm 1: Flow Matching Training

```
for each batch:
    x_1 = sample_data() # from dataset
    x_0 = torch.randn_like(x_1) # from N(0, I)
    t = torch.rand(batch_size, 1) # from U(0, 1)
    x_t = (1 - t) * x_0 + t * x_1 # interpolation
    v_target = x_1 - x_0 # target velocity
    v_pred = model(x_t, t) # network prediction
    loss = MSE(v_pred, v_target)
    loss.backward()
    optimizer.step()
```

6.5 Complete Sampling Algorithm

Algorithm 2: Flow Matching Sampling (Euler)

```
x = torch.randn(n_samples, dim) # start from noise
dt = 1.0 / n_steps
for i in range(n_steps):
    t = i * dt
    v = model(x, t)
    x = x + v * dt # Euler step
# x is now a generated sample
```

7 Advanced: Reflow for One-Step Generation

After training a basic flow matching model v_1, you can straighten the trajectories further via the reflow procedure.

7.1 Generating Coupled Pairs

Step 1: Generate coupled pairs using v_1

```
# Generate noise samples
x_0 = torch.randn(N, dim)

# Solve ODE forward from t=0 to t=1 using learned v_1
x = x_0.clone()
dt = 1.0 / n_steps
for i in range(n_steps):
    t = i * dt
    x = x + v_1(x, t) * dt
z_1 = x # coupled output

# Store (x_0, z_1) pairs -- these are now correlated
```

7.2 Retraining on Coupled Pairs

Step 2: Retrain v_2 on coupled pairs

```
for each batch:
    # Use COUPLED pairs instead of independent samples
    x_0, z_1 = sample_coupled_pairs()
    t = torch.rand(batch_size, 1)
    x_t = (1 - t) * x_0 + t * z_1
    v_target = z_1 - x_0
    v_pred = model_v2(x_t, t)
    loss = MSE(v_pred, v_target)
    ...
```

7.3 One-Step Generation After Reflow

After sufficient reflow rounds, the trajectories are approximately straight, enabling:

$$x_{\text{generated}} = x_0 + v_k(x_0, 0), \qquad x_0 \sim \mathcal{N}(0, \mathbf{I}) \tag{38}$$

This replaces the entire multi-step ODE integration with a single function evaluation.

8 Reading Order and Resources

8.1 Recommended Reading Order

1. **Start here:** `THEORY/GUIDE_Flow_Matching_for_ToyDiffusion.md`
   Practical, code-focused, maps directly to ToyDiffusion.

2. **Intuition:** Cambridge MLG Blog
   https://mlg.eng.cam.ac.uk/blog/2024/01/20/flow-matching.html
   Builds from CNFs → FM → CFM → OT paths.

3. **Interactive visuals:** Diffusion Meets Flow Matching
   https://diffusionflow.github.io/
   Mathematical equivalence shown visually.

4. **Paper 1:** Flow Matching (Lipman et al., 2022)
   https://arxiv.org/abs/2210.02747
   Read Sections 1–3 for the math.

5. **Paper 2:** Rectified Flow (Liu et al., 2022)
   https://arxiv.org/abs/2209.03003
   Read Sections 1–3 for the reflow procedure.

6. **Implementation companion:** Flow Matching Guide and Code (Lipman et al., 2024)
   https://arxiv.org/abs/2412.06264
   Comprehensive tutorial with PyTorch code.

7. **MIT Course:** Flow Matching and Diffusion Models (2026)
   https://diffusion.csail.mit.edu/docs/lecture-notes.pdf

8.2 Code Repositories

Repo	What	Link
RectifiedFlow	Official PyTorch impl.	https://github.com/gnobitab/RectifiedFlow
ToyDiffusion	Your original DDPM project	https://github.com/ThiagoLira/ToyDiffusion

Flow Matching Cheat Sheet

Quick Reference Card

Training:

$$
\begin{aligned}
x_1 &\sim p_{\text{data}} && \text{sample data} \\
x_0 &\sim \mathcal{N}(0, \mathbf{I}) && \text{sample noise} \\
t &\sim U(0,1) && \text{sample time} \\
x_t &= (1-t)\, x_0 + t\, x_1 && \text{interpolate} \\
v &= x_1 - x_0 && \text{target velocity} \\
\mathcal{L} &= \|v_\theta(x_t, t) - v\|^2 && \text{MSE loss}
\end{aligned}
$$

Sampling (Euler, N steps):

$$
\begin{aligned}
& x \sim \mathcal{N}(0, \mathbf{I}), \qquad \Delta t = 1/N \\
& \text{for } i = 0, \ldots, N-1 : \\
& \qquad t = i \cdot \Delta t \\
& \qquad x \leftarrow x + v_\theta(x, t) \cdot \Delta t \\
& \text{return } x
\end{aligned}
$$

What the network predicts:

$$\text{Input: } (x_t, t) \quad \longrightarrow \quad \text{Output: } v \in \mathbb{R}^d \text{ (velocity, same dim as } x_t)$$

That's it. No βs. No αs. No posterior. Just velocity.

References

[1] J. Ho, A. Jain, P. Abbeel. *Denoising Diffusion Probabilistic Models.* NeurIPS 2020. https://arxiv.org/abs/2006.11239

[2] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, M. Le. *Flow Matching for Generative Modeling.* ICLR 2023. https://arxiv.org/abs/2210.02747

[3] X. Liu, C. Gong, Q. Liu. *Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow.* ICLR 2023 (Spotlight). https://arxiv.org/abs/2209.03003

[4] Y. Lipman, M. Havasi, P. Holderrieth, N. Shaul, M. Le, B. Karrer, R. T. Q. Chen, D. Lopez-Paz, H. Ben-Hamu, I. Gat. *Flow Matching Guide and Code.* 2024. https://arxiv.org/abs/2412.06264

[5] A. Vaswani, N. Shazeer, N. Parmar, et al. *Attention Is All You Need.* NeurIPS 2017.

[6] J. Song, C. Meng, S. Ermon. *Denoising Diffusion Implicit Models (DDIM).* ICLR 2021. https://arxiv.org/abs/2010.02502